

Implicit and Explicit Conversions

Document number: P0705R0

Date: 2017-06-19

Audience: LEWG

Reply-to: Tony Van Eerd. conversions at forecode.com

***Note:** This paper intentionally left unfinished - I would like to get feedback before continuing.*

Three Types of Conversions

Let's be explicit about the different conversions:

1. Implicit Constructor and/or Implicit Cast

```
struct Int
{
    Int(int);
    operator int() { return 17; }
};

Int J = 17; // Int(17) constructor
int k = J;  // J.operator int()
```

2. Explicit Constructor and/or Explicit Cast

```
struct Ent
{
    explicit Ent(int) {};
    explicit operator int() { return 23; }
};

// three "spellings" for calling Ent(int)
Ent E = Ent(17);
Ent F = (Ent)17;
Ent G = static_cast<Ent>(17);

// three "spellings" for calling operator int()
int x = int(E);
int y = (int)E;
int z = static_cast<int>(E);
```

P.S. I find it weird and unfortunate that casting and construction are the same. ie:

```
class Image {
    Image(char const * filename) { ... }
};
Image x = (Image)"foo.png"; // cast a string to an Image???
```

3. “Named” Conversion

```
struct Nnt
{
    Nnt();
    // inside class...
    static Nnt from_int(int) { return Nnt(); }
    int to_int() { return 31; }
};
```

```
// ...or free functions:
Nnt to_Nnt(int) { return Nnt(); }
int to_int(Nnt) { return 13; }

Nnt N = Nnt::from_int(5);
Nnt M = to_Nnt(7);

int w = N.to_int();
int v = to_int(N); // more generic
```

Which should be used When, and Why?

When determining whether to use implicit, explicit, or named conversions, there are some aspects to consider. Answering these questions can help decide which type of conversion should be used.

If we are to form general LEWG guidelines, we need to decide which of these are worth considering, and to what extent.

Same “Platonic” Thing

Many C++ objects attempt to represent objects or ideas outside of the language. ie, things such as “numbers” and “dates”. For example, both `int` and `long` attempt to represent numbers and their operations, but can’t represent all numbers. The term “platonic” is used here in reference to Plato’s [Theory of Forms](#). When converting from one type to another, it is important to ask whether both types represent the same platonic thing, and thus whether the conversion is “natural”. Converting from `int` to `long` both represent numbers, so is “natural”. But `int` and `string` do not represent the same platonic thing.

Some types are obviously not attempting to model the same “thing”. An `Image` class may have a constructor like `Image(file_system::path const & file)`, but obviously an `Image` does not attempt to model the same thing as `file_system::path`.

This is probably the number one criteria for deciding implicit vs explicit - is it the same platonic thing.

Implicit conversions must just “do the right thing” - ie `std::chrono::seconds` to `minutes`. Whereas `int` to `seconds` is explicit because `int` models a `Number`, and `seconds` models a `Duration` - not the same Thing.

Information Fidelity

When considering the platonic form, we also need to consider how well the platonic is represented and preserved. Is there information loss? `int` represents fewer numbers than `long` does (typically). `char *` cannot have embedded nulls, `string_view` and `string` can. `string` cannot separate “empty” from “null/invalid”, `char *` (null vs “”) and `string_view(.data()==nullptr vs .length()==0)` can (although you may argue whether `string_view.data()` should be used that way).

Performance

Performance is always important in C++. A conversion could have both time and space performance considerations. Most programmers may expect implicit conversions to take “no time”. Hiding lots of code behind overloaded operators is a common complaint about C++. Maybe **complexity** could be a separate category - developers expect implicit conversions to be “simple”. You don’t want to be forced to trace into implicit conversions when debugging - they are too easy to skip over.

Note, however, that things like `string(string const &)`, `vector(vector const &)`, etc are implicit copy constructors, and they have performance impacts.

Failure

Does the conversion throw? Does it allocate memory? Can it fail?

Note, however, that things like `string(string const &)`, `vector(vector const &)`, etc are implicit copy constructors, and yet they throw.

Danger

Some conversions are platonic, fast, don't throw, etc, yet are dangerous. ie converting a `std::string` to a `char *`. We could make this implicit, but instead made it a named conversion `c_str()`. Why? Because the pointer could be left dangling - it is too easy to be dangerous.

Code Review

This is somewhat a collection of the other properties. If the conversion is dangerous or lossy or throws, etc, then maybe you want to be able to easily *police* it during code reviews, and easily spot it when debugging. Implicit conversions are hardest to spot, named conversions easiest.

Generic Code

How would this work in generic code?

Each type of conversion interacts differently with generic code. Be careful with explicit casts when you don't actually know the types involved (ie due to templates) - you don't know what you are being explicit about. *User code should avoid using explicit casts/constructors - if all code is explicit, explicit == implicit.*

```
template <class Duration1, class Duration2>
nanoseconds average(Duration1 d1, Duration2 d2)
{
    // whoops: explicit ctor used
    auto ns = nanoseconds{d1 + d2};
    return ns/2;
}
```

```
// much later...
int i = ...;
int j = ...;
nanoseconds a = average(i, j);
```

vs

```
template <class Duration1, class Duration2>
nanoseconds average(Duration1 d1, Duration2 d2)
{
    nanoseconds ns = d1 + d2;
    return ns/2;
}
```

If you know you want to work with disparate (not same-platonic-thing) types, pick a named function (like `to_string`) and make it a requirement on your concept. This aspect is more about what you expect templates to look like, than what your conversion looks like.

Named free-function conversions can be good extension points.

However, they do require agreement on the name - which is fine for STL, but harder for independent libraries. ie my library called it `to_int` but your library called it `to_integer`. De facto standards - *plural*. :-)

Note: Extensions points in STL have their own problems. ADL, using `std::swap`. etc.

Modifying Existing Classes

Constructors and casts are not free functions. Named conversions can be. Thus only named conversions are possible for classes that you can't modify. We *can* modify existing STL classes, but we need to be **careful** about code breakage. We can, of course, modify newly-proposed STL classes.

If In Doubt

Although the language rule is implicit-by-default, as a coding guideline, I think the default should be explicit. ie suggestion:

All new STL classes should have explicit constructors unless there is _motivation to make it otherwise._

Given the above considerations, we can express this in a table. *There's always a table*

How to read this table.

Think of a conversion. Answer the questions along the left column. The *rightmost* column that gets a "check" for your conversion is the type of conversion you should choose (except the 'generic code' column). Thus only choose implicit if ALL checks are in the implicit column. "generic code" exception: you could choose Named *in addition to* Implicit/Explicit if you are choosing Named just as an extension point. (ie string can still have a `to_string` function). And yes, the differences between Explicit and Named are not always cut and dry.

Consideration	Your Class?	Your Class?	Your Class?
same platonic thing?	yes	no[1]	-
info fidelity	no loss	some loss	more loss
performance penalty?	little/no	some	yes
throws?	noexcept?/rarely?/ same as copyctor?	yes	-
danger? (dangling, etc)	no	yes	-
code review?	no need	self-policed[2]	greppable / policeable
generic code?	strict	less strict	"extension point"
can modify class?	yes	yes	no
are you sure?	yes	no	-
Result	Implicit ctor/cast	Explicit cast/ctor	Named

1. If not the same platonic thing, you can have an explicit constructor, but you shouldn't have a cast at all
2. 'self-policed' - Explicit conversions are more for situations where you want the developer to stop for a second and think about the conversion, but have enough faith in the average developer to make a good choice, and don't feel it typically needs much further policing. You can see it in a code review, but harder to grep for.

How are we (std::) doing?

- `string(char *)` should be explicit? (can throw). But sooo convenient. But now we have `string_view` - doesn't throw.
- `chrono :-)` `chrono` is an excellent example of thoughtful conversions
- `std::to_string`
- `std::byte` to `_integer`

Elsewhere

- `boost lexical_cast`
- `boost units`
- `foonathan's typesafe library`

Other Things to Consider

We need guidelines that also cover these issues:

- **Constructor or cast operator?** - see `string_view` vs `string` - which should depend on which. If can convert both ways, both conversions should be in *same* class, not one in each. (ie no circular dependencies)
- **beware of ambiguity** - if `string_view` constructed from `string` and `string` had a cast to `string_view` we'd get ambiguity errors
- **member function or free function?**
- **extension point?**
- Use “`_cast`” in name?

Acknowledgements

Howard Hinnant knows this better than I do, and gave me lots of input and examples. Such as `chrono`, which is one big example of explicit vs implicit. I also appreciate the encouragement I received from Jeffrey, Gaby, and others.